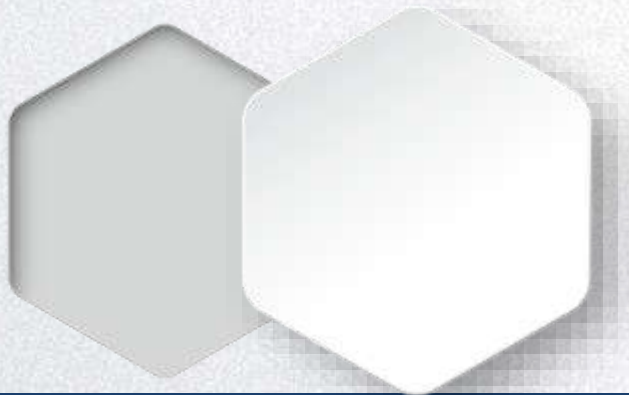




《人工智能与Python程序设计》——PyTorch基础



人工智能与Python程序设计 教研组



提纲



PyTorch基础

- ☐ 张量
- ☐ PyTorch nn工具箱
- ☐ PyTorch实现线性回归
- ☐ 计算图
- ☐ 作业: PyTorch实现逻辑回归

总结



- Pytorch是简单易学的人工智能平台
- 两个主要的组成部分
 - Tensor
 - AutoGrad
- 上次课
 - 以数学运算为背景，介绍了如何利用Pytorch进行简单计算和求导

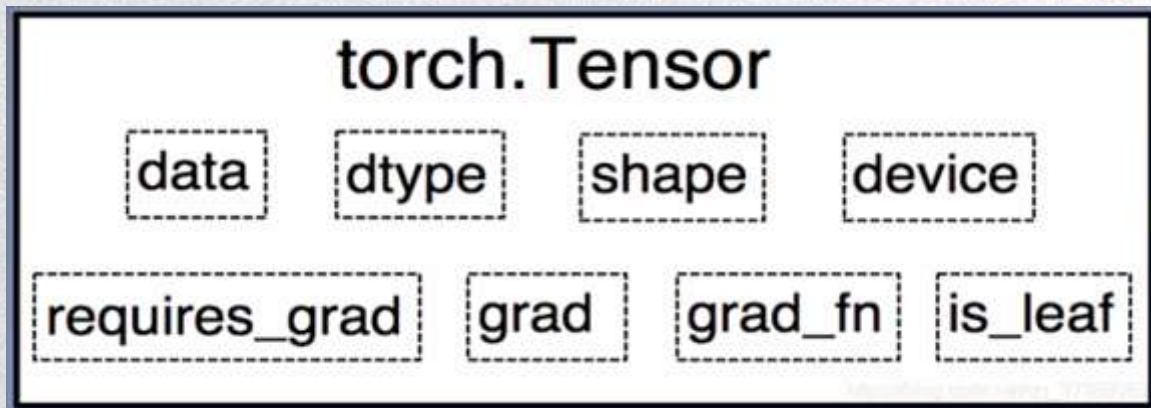
张量复习

- 张量：多维数组
- 标量—向量—矩阵——张量



张量与自动求导

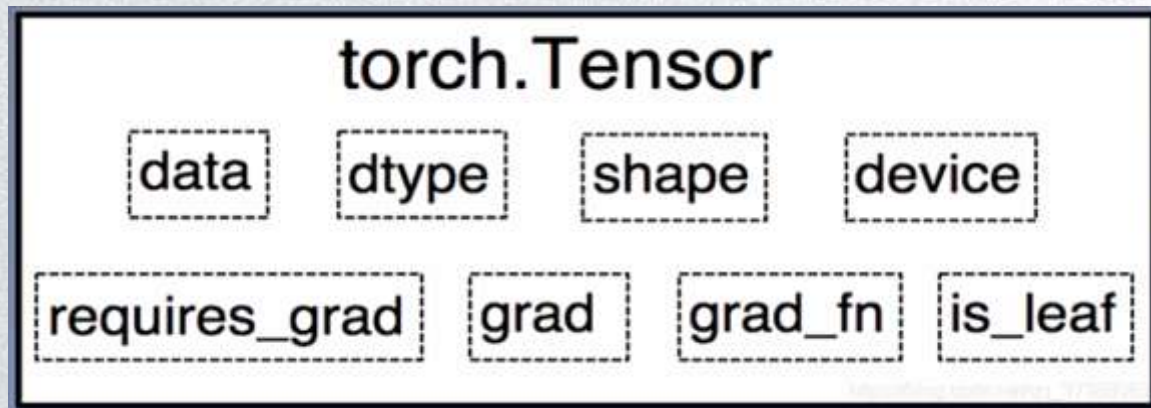
- data: 被封装的tensor
- grad: data的梯度
- grad_fn: 创建tensor的function
- requires_grad: 指示是否需要梯度
- is_leaf: 指示是否是叶子结点





张量的属性

- dtype: 张量的数据类型, 如torch.FloatTensor, torch.cuda.FloatTensor;
 - 两个数据类型之间不同之处在于后者多了一个cuda, cuda表示数据是放到GPU上的。
- shape: 张量的形状, 如 (64, 3, 224, 224)
- device: 张量所在设备, GPU/CPU



例子

```
import torch as t
A = t.empty(3, 4, requires_grad=True)
print(A.data)

tensor([[1.7266e-07, 1.2973e-11, 5.3132e-08, 2.6705e+23],
        [5.3686e-05, 4.1292e-05, 2.0431e+20, 8.5452e-07],
        [4.1429e-11, 7.1450e+31, 4.1418e-41, 0.0000e+00]])
```

```
print(A.dtype)
```

```
torch.float32
```

```
print(A.shape)
```

```
torch.Size([3, 4])
```

```
print(A.device)
```

```
cpu
```

```
print(A.requires_grad)
```

```
True
```

```
print(A.grad)
```

```
None
```

```
print(A.grad_fn)
```

```
None
```

```
print(A.is_leaf)
```

```
True
```



nn工具箱



- torch.nn是专门为深度学习设计的工具箱
- 主要包括：
 - 参数类 **Parameters**
 - 模型基类 **Module**
 - 具有不同功能的网络层：卷积、池化、全连接、回归等
 - 非线性激活
 - 损失函数
 - 辅助函数

torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions



线性回归问题回顾

- 假设目标值与特征之间线性相关，即因变量与自变量满足一个多元一次方程。
- 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$
- 将特征 $\mathbf{x}$ 和对应的目标 y 建模为多元一次方程：

$$\hat{y} = \mathbf{x}\mathbf{w} + b * 1$$

其中， $\mathbf{w}$ 和 b 为模型的参数。

- $\mathbf{x}$ 为行向量， $\mathbf{w}$ 为列向量
- 为了简化符号表示，在 $\mathbf{x}$ 后面加上一维值恒为1的特征， $\mathbf{x} \leftarrow [\mathbf{x}, 1]$ ，同时 $\mathbf{w}$ 增加一维，则可以把 b 融入 $\mathbf{w}$

$$\hat{y} = \mathbf{x}\mathbf{w}$$

- 为了学习这两个参数，根据已知的训练样本点（自变量 $\mathbf{x}$ 和因变量 y 都是已知的）拟合这个多元一次方程，求解参数。



基于PyTorch的线性回归梯度计算

```
import numpy as np
import torch as t
import matplotlib.pyplot as plt
```

```
def __calc_gradient(self, x, y):
```

类的方法: 计算对w的梯度

```
#numpy version
```

```
N = x.shape[0]
diff = (x.dot(self.w) - y)
grad = x.T.dot(diff)
d_w = (2 * grad) / N
```

```
#torch version
```

```
x_tensor = t.tensor(x)
y_tensor = t.tensor(y)
w_tensor = t.tensor(self.w, requires_grad=True)
loss = t.sum(t.pow(x_tensor.mm(w_tensor) - y_tensor, 2)) / N
loss.backward()
d_w = w_tensor.grad.numpy()
return d_w
```

- 用PyTorch实现的要点
 - 引入torch包
 - 把numpy变量转为Tensor变量, 注意只对w求导
 - 用pyTorch中的方法实现损失函数
 - 调用backward()
 - 返回对应的梯度值
- 优点
 - 不用显式求解和实现梯度公式
 - 在一般情况下, 梯度函数比损失函数更加难以实现



线性回归

- 继承 `nn.Module`, 实现线性回归

```
class LinearRegression(nn.Module):  
    def __init__(self, in_dim): #构造函数, 需要调用nn.Module的构造函数  
        super().__init__() #等价于nn.Module.__init__()  
        self.w=nn.Parameter(torch.randn(in_dim+1, 1))  
  
    def forward(self, x):  
        x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
        x = x.matmul(self.w)  
        return x
```


Module



- Module是torch.nn 的**核心数据结构**，是一个抽象的概念，既可以表示神经网络的某个层(layer)，也可以表示一个包含很多层的神经网络。
- 是所有神经网络模块的基类
- Module也可以包含其它modules。
- 成员变量：
 - training (bool)：指示这个module是训练还是测试模式
- 线性回归模型可看作是一个简单的神经网络



自定义线性回归Module

- 自定义层LinearRegression 继承nn.Module, 并且在其构造函数中调用nn.Module 的构造函数, 即

super().__init__() 或 nn.Module.__init__()

```
def __init__(self, in_dim): #构造函数, 需要调用nn.Module的构造函数  
    super().__init__()      #等价于nn.Module.__init__()
```

通过对x增加一个恒为1的特征:

```
self.w=nn.Parameter(torch.randn(in_dim+1, 1))
```

参数类



Parameter

A kind of Tensor that is to be considered a module parameter.

CLASS `torch.nn.parameter.Parameter`

[SOURCE]

A kind of Tensor that is to be considered a module parameter.

Parameters are `Tensor` subclasses, that have a very special property when used with `Module`s - when they're assigned as `Module` attributes they are automatically added to the list of its parameters, and will appear e.g. in `parameters()` iterator. Assigning a `Tensor` doesn't have such effect. This is because one might want to cache some temporary state, like last hidden state of the RNN, in the model. If there was no such class as `Parameter`, these temporaries would get registered too.

Parameters

- **data** (*Tensor*) – parameter tensor.
- **requires_grad** (*bool*, optional) – if the parameter requires gradient. See [Excluding subgraphs from backward](#) for more details. Default: `True`

参数类



- torch.nn.parameter.Parameter
- Tensor的子类，一种特殊的张量，module的参数
 - 当Parameter被指定为Module的属性时，自动注册在parameters() 迭代器
- 属性：
 - data (Tensor)
 - requires_grad (bool, optional) : 默认是True

参数类



迭代器

迭代是Python最强大的功能之一，是访问集合元素的一种方式。

迭代器是一个可以记住遍历的位置的对象。

迭代器对象从集合的第一个元素开始访问，直到所有的元素被访问完结束。迭代器只能往前不会后退。

迭代器有两个基本的方法：`iter()` 和 `next()`。

字符串，列表或元组对象都可用于创建迭代器：

实例(Python 3.0+)

```
>>> list=[1,2,3,4]
>>> it = iter(list)    # 创建迭代器对象
>>> print (next(it))   # 输出迭代器的下一个元素
1
>>> print (next(it))
2
>>>
```



自定义线性回归Module

- 成员函数: `forward()`, 实现前向传播过程, 其输入可以是一个或多个Tensor, 对x的任何操作也必须是Tensor支持的操作。

forward(self, x, [y, ...])

- 定义每次调用Module实例时执行的计算, 网络/层前向计算过程
- 所有子类/继承类都应重写此函数

```
def forward(self, x):  
    x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
    x = x.matmul(self.w)  
    return x
```

- 无需写反向传播函数, `nn.Module`能够利用`autograd`自动实现反向传播。



自定义线性回归Module

- 使用时，首先实例化一个LinearRegression类

例如：

```
def testLRmodel(in_dim, data_size = 2):  
    layer = LinearRegression(in_dim)  
    input=torch.randn(data_size,in_dim)  
    output=layer(input) #前向传播 执行forward()  
    print(output)  
    for parameter in layer.parameters():  
        print(parameter)
```

- 直观上可以将layer看成数学概念中的函数，调用model(input)即可得到input 对应的前向计算（forward）结果。

等价于**layer.__call__(input)**。

pytorch在nn.Module中实现了__call__方法，**并且在__call__方法中调用了forward函数**



__call__()

- __call__()的作用是使实例能够像函数一样被调用

```
1 class X(object):
2     def __init__(self, a, b, range):
3         self.a = a
4         self.b = b
5         self.range = range
6     def __call__(self, a, b):
7         self.a = a
8         self.b = b
9         print('__call__ with ({}, {}) '.format(self.a, self.b))
```

```
1 >>> xInstance = X(1, 2, 3)
2 >>> xInstance(1,2)
3 __call__ with (1, 2)
```



自定义线性回归Module

- Module中的可学习参数可以通过成员函数parameters() 返回。

```
22  for parameter in layer.parameters():  
23      print(parameter)
```

```
Parameter containing:  
tensor([[1.2220],  
        [1.0318],  
        [0.3946],  
        [0.3981]], requires_grad=True)
```




Pytorch模型训练

- 实现Linear_Model类，实现其成员函数对线性回归模型进行训练和测试
- 初始化：创建模型和优化器，初始化线性模型和优化器超参数

```
class Linear_Model():  
    def __init__(self, in_dim):  
          
        创建模型和优化器，初始化线性模型和优化器超参数  
          
        self.learning_rate = 0.01  
        self.epoches = 10000  
        self.model = LinearRegression(in_dim) #torch.nn.Linear(in_dim,1)  
        self.optimizer = torch.optim.SGD(self.model.parameters(), lr=self.learning_rate)  
        self.loss_function = torch.nn.MSELoss()
```

- 线性回归模型参数已在实例化LinearRegression类时用正态分布随机初始化

```
class LinearRegression(nn.Module):  
    def __init__(self, in_dim): #构造函数，需要调用nn.Module的构造函数  
        super().__init__() #等价于nn.Module.__init__()  
        self.w=nn.Parameter(torch.randn(in_dim+1, 1))
```



Pytorch优化器



- Pytoch中用来优化模型权重的类: **torch.optim.Optimizer**
- 各种常用的优化器都是Optimizer这个基类的子类
- *CLASS torch.optim.Optimizer(params, defaults)*
- 参数: params的数据类型有两种:
 - 通常把model.parameters()传进去



Pytorch优化器

- 随机梯度下降优化器

```
self.optimizer = torch.optim.SGD(self.model.parameters(), lr=self.learning_rate)
```

- 参数：模型的parameters()方法返回模型参数iterator
lr: 学习率

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \left(\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} \bigg|_{\mathbf{w}^t} \right)^T$$

Pytorch优化器

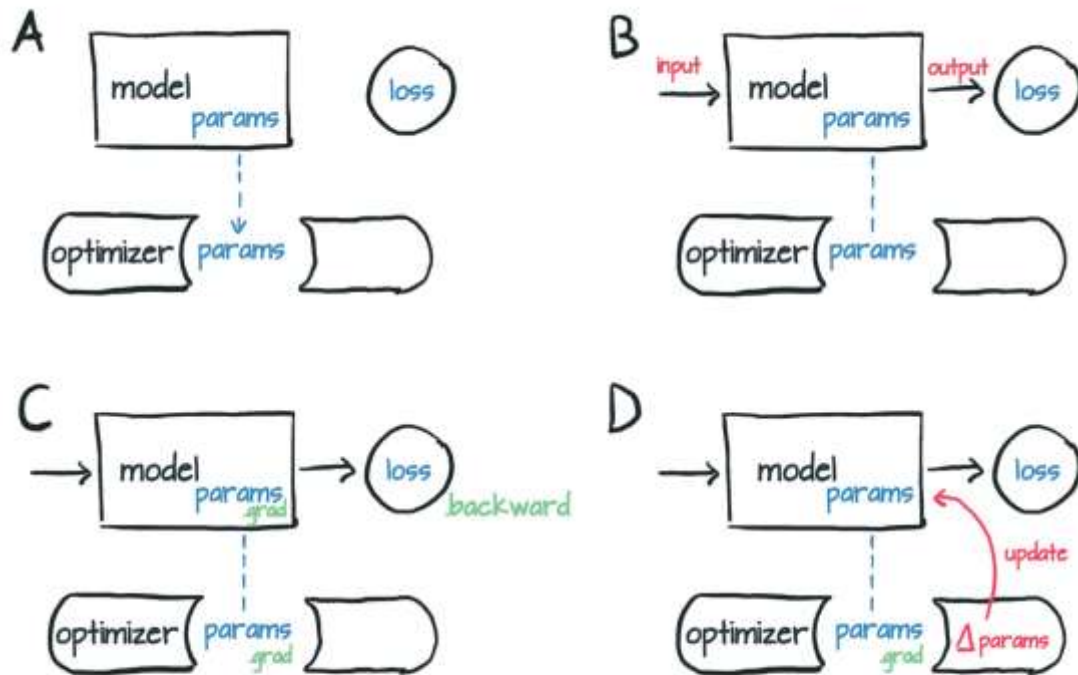


Figure 4.10 Conceptual representation of how an optimizer holds a reference to parameters (A), and after a loss is computed from Inputs (B), a call to `.backward` leads to `.grad` being populated on parameter (C). At that point, the optimizer can access `.grad` and compute the parameter updates (D).



Pytorch损失函数

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- 均方误差损失 (nn.MSELoss) 实例化:

```
self.loss_function = torch.nn.MSELoss()
```

- MSELoss: 输入两个tensor (所有样本的预测值、所有样本的真值), 返回均方误差



Pytorch模型训练

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epochs):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
    if epoch % 500 == 0:  
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

模型使用当前参数的
对训练样本进行预测

计算预测值和真值的
MSE损失

Pytorch模型训练



```
def train(self, x, y):
```

```
    """
```

训练模型并保存参数

输入:

model_save_path: saved name of model

x: 训练数据

y: 回归真值

返回:

losses: 所有迭代中损失函数值

```
    """
```

```
    losses = []
```

```
    for epoch in range(self.epochs):
```

```
        prediction = self.model(x)
```

```
        loss = self.loss_function(prediction, y)
```

```
        self.optimizer.zero_grad()
```

```
        loss.backward()
```

```
        self.optimizer.step()
```

```
        losses.append(loss.item())
```

```
    if epoch % 500 == 0:
```

```
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

torch.Tensor

data

dtype

shape

device

requires_grad

grad

grad_fn

is_leaf

pytorch中的backward()

函数的计算，梯度是
被积累的而不是被替
换掉

所有参数的梯度置0



Pytorch模型训练

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epoches):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
    if epoch % 500 == 0:  
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

计算梯度



Pytorch模型训练

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epochs):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
    if epoch % 500 == 0:  
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \left(\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} \bigg|_{\mathbf{w}^t} \right)^T$$

所有参数更新一次



Pytorch模型训练

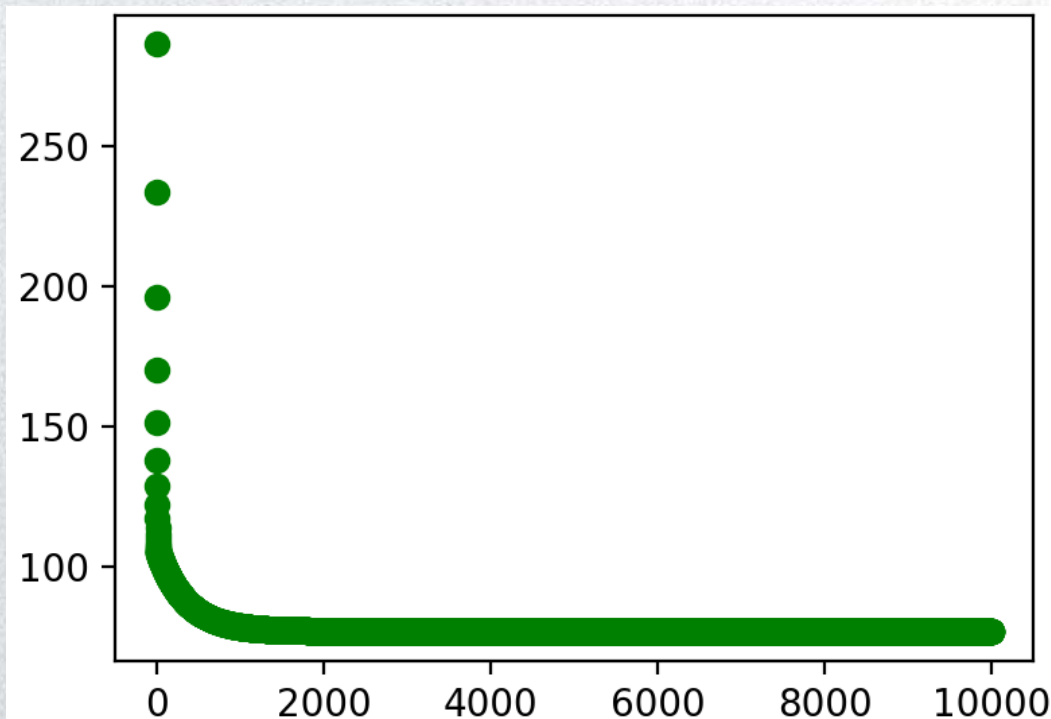
```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epoches):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
        if epoch % 500 == 0:  
            print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

每500轮打印一次损失值

Pytorch模型训练



- Loss随着迭代次数的变化





Pytorch模型测试

```
def test(self, x, y, if_plot = True):  
    """  
    用保存或训练好的模型做测试  
    输入:  
        model_path: 训练好的模型的保存路径, e.g., "linear.pth"  
        x: 测试数据  
        y: 测试数据的回归真值  
    返回:  
        prediction: 测试数据的预测值  
    """  
    prediction = self.model(x)  
    testMSE = self.loss_function(prediction, y)  
  
    if if_plot and x.shape[1]==1:  
        plt.figure()  
        plt.scatter(x.numpy(), y.numpy())  
        plt.plot(x.numpy(), prediction.numpy(), color="r")  
        plt.show()  
  
    return prediction, testMSE
```

用模型对测试数据进行预测

计算预测结果和测试数据真值的MSE损失



Pytorch模型测试

```
def test(self, x, y, if_plot = True):  
    """  
    用保存或训练好的模型做测试  
    输入:  
        model_path: 训练好的模型的保存路径, e.g., "linear.pth"  
        x: 测试数据  
        y: 测试数据的回归真值  
    返回:  
        prediction: 测试数据的预测值  
    """  
  
    prediction = self.model(x)  
    testMSE = self.loss_function(prediction, y)  
  
    if if_plot and x.shape[1]==1:  
        plt.figure()  
        plt.scatter(x.numpy(), y.numpy())  
        plt.plot(x.numpy(), prediction.numpy(), color="r")  
        plt.show()  
  
    return prediction, testMSE
```

用模型对测试数据进行预测

计算预测结果和测试数据真值的MSE损失



Pytorch模型测试

- `in_dim = 3`

```
w真值:tensor([[ -0.5206],  
              [-3.0018],  
              [ 1.4464]])
```

```
w Parameter containing:  
tensor([[ -0.6389],  
        [-3.8588],  
        [ 0.9694],  
        [18.1736]], requires_grad=True)
```

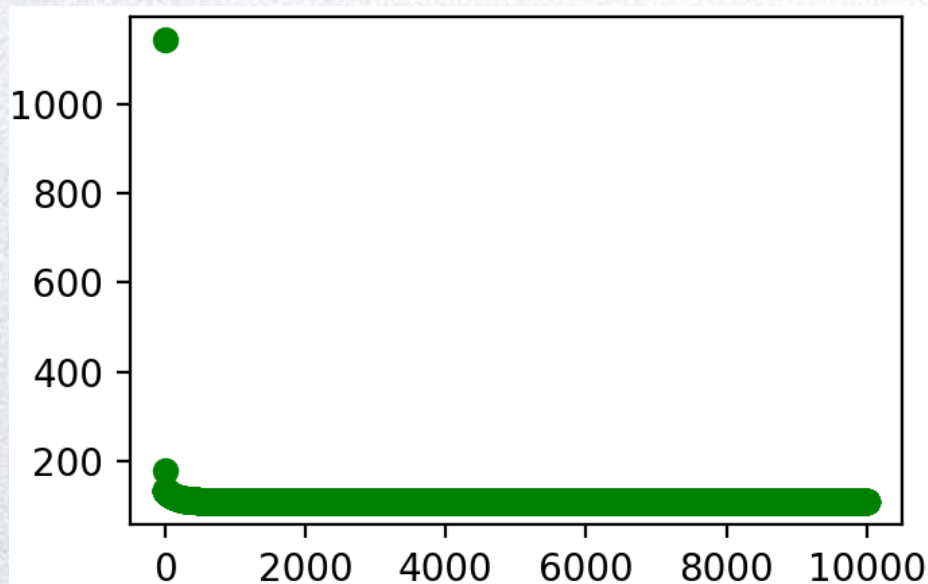
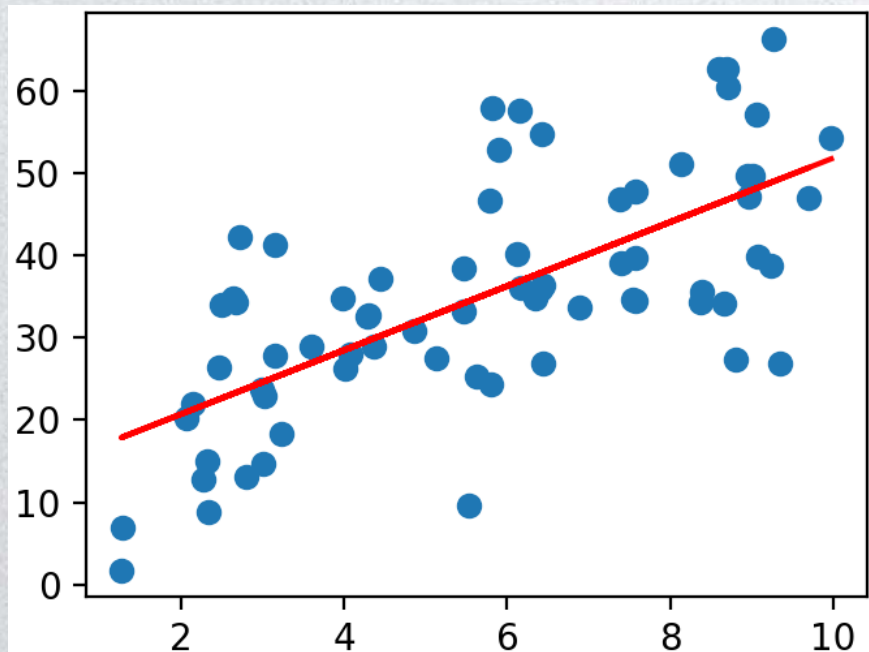
测试集上MSE损失值:130.86734008789062

保存的模型在测试集上MSE损失值:130.86734008789062



Pytorch模型训练测试

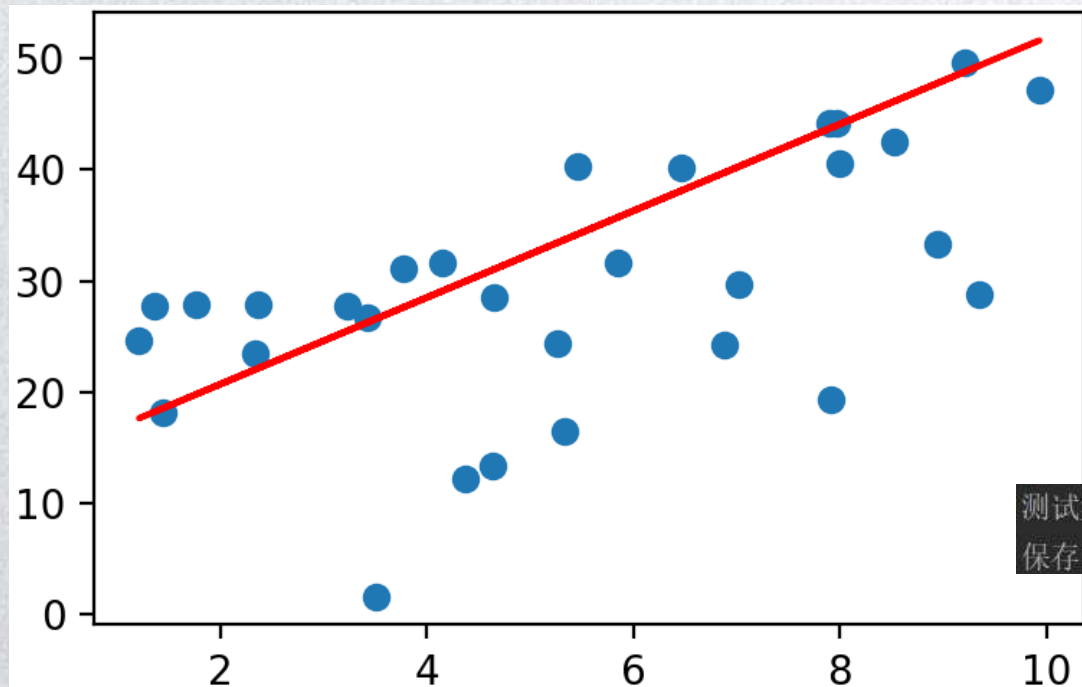
- $\text{in_dim}=1$, 训练:





Pytorch模型训练测试

- $\text{in_dim}=1$, 测试:



w真值: `tensor([[4.1975]])`

w Parameter containing:
`tensor([[3.9004],
[12.9037]], requires_grad=True)`

测试集上MSE损失值: 119.6384048461914

保存的模型在测试集上MSE损失值: 119.6384048461914



Pytorch和numpy实现线性回归对比

```
def fit(self, x, y):  
    """
```

类的方法：训练函数

参数 自变量：x

参数 因变量：y

返回每一次迭代后的损失函数

```
    """  
  
    for i in range(self.max_iter):  
        self.__train_step(x, y)  
        y_pred = self.predict(x)  
        self.loss_arr.append(self.loss(y, y_pred))
```

```
def __train_step(self, x, y):  
    """
```

类的方法：单步迭代，即一次迭代中对梯度进行更新

```
    """  
  
    d_w, d_b = self.__calc_gradient(x, y)  
    self.w = self.w - self.lr * d_w  
    self.b = self.b - self.lr * d_b  
    return self.w, self.b
```

```
def __calc_gradient(self, x, y):  
    """
```

类的方法：分别计算对w和b的梯度

```
    """  
  
    d_w = np.mean(2 * (x * self.w + self.b - y) * x)  
    d_b = np.mean(2 * (x * self.w + self.b - y))  
    return d_w, d_b
```

PyTorch

```
def train(self, x, y):  
    """
```

训练模型并保存参数

输入：

model_save_path: saved name of model

x: 训练数据

y: 回归真值

返回：

losses: 所有迭代中损失函数值

```
    """
```

```
    losses = []
```

```
    for epoch in range(self.epochs):
```

```
        prediction = self.model(x)
```

```
        loss = self.loss_function(prediction, y)
```

```
        self.optimizer.zero_grad()
```

```
        loss.backward()
```

```
        self.optimizer.step()
```

```
        losses.append(loss.item())
```

```
    if epoch % 500 == 0:
```

```
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

Module



- Torch.nn几乎涵盖了所有常用的层和网络结构，比如卷积层，池化层，线性层，回归层，Transformer层等
- 通过Sequential等容器可以很容易地构造各种深度神经网络结构，就像搭积木一样
- Torch.nn同样包含了几乎所有常见的损失函数，如nn.CrossEntropyLoss, nn.MSELoss等
- 将在后续课程具体实例中介绍



大作业3：用pytorch实现逻辑回归

- 仿照上述线性回归的代码，用pytorch实现逻辑回归
- 给定了.py文件的模板，在需要写程序的地方都有注释
- 要实现的任务包括：
 - LogisticRegression的__init__和 forward函数
 - Logistic_Model类的__init__、train和test函数
- 提交期限：2周
- 提交平台：obe

```
class LogisticRegression(nn.Module):  
    def __init__(self, in_dim):  
        """  
        ***** 请在此处输入你的代码*****  
        定义模型参数w, 以及非线性sigmoid函数  
        """  
        super().__init__() # 等价于nn.Module.__init__()  
  
    def forward(self, x):  
        """
```

```
class Logistic_Model():  
    """  
    模型训练与测试类  
    """  
    def __init__(self, in_dim):...  
  
    def train(self, x, y):...  
  
    def test(self, x, y):...
```



谢谢！



计算图

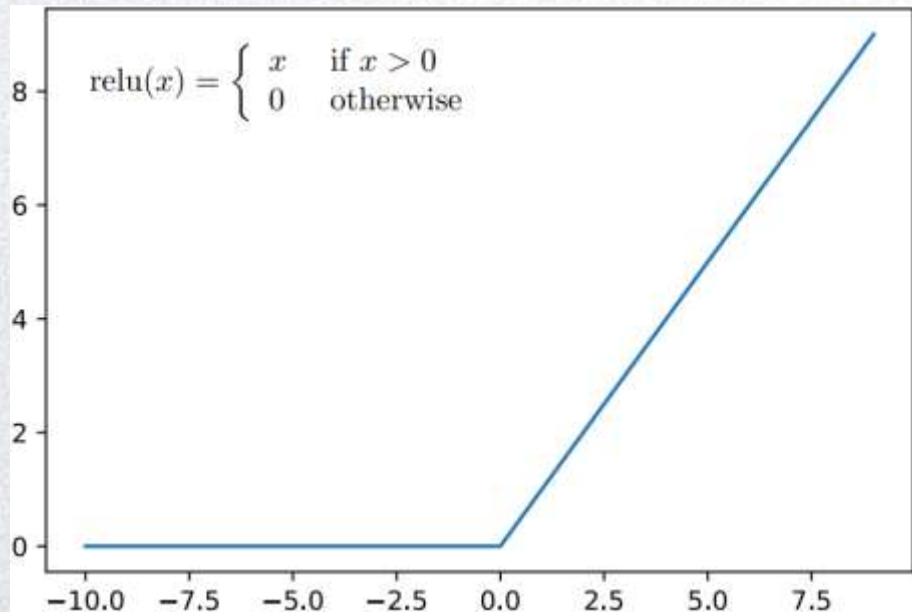
- Tensor的属性中，4个与数据相关，4个与梯度求导相关
- Pytorch是如何实现自动求导的？
- 深度学习和PyTorch中，将深度神经网络看作计算图
- 节点表示数据
 - Tensors; Parameters
 - 早期版本: Variables
- 边表示计算
 - 加减乘除
 - 卷积、池化

计算图

- 例如：对激活函数

$$a(x, w, b) = \text{relu}(w \cdot x + b)$$

- ReLU = Rectified Linear Unit
 - 可能是现在神经网络中最常用的激活函数
 - 原因（之一）：数值过大或者过小，sigmoid, tanh的导数接近于0
 - 练习：计算Relu函数的梯度





关于Relu函数

- Relu函数求导

- DNE: does not exist
- 在0处不可导
- 为了方便, 令其导数为:

$$f'(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x > 0 \\ \text{DNE} & \text{if } x = 0 \end{cases}$$

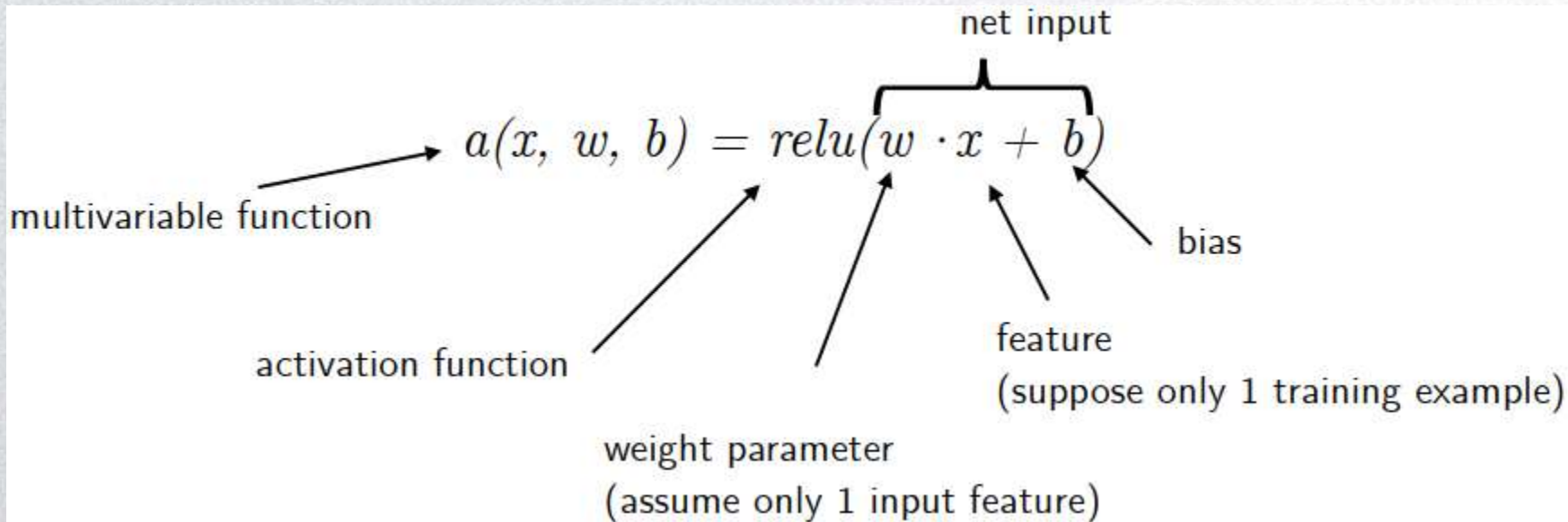
$$f'(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$$

- Relu函数其它优点

- 防止梯度消失
- 梯度简单易计算, 使网络训练更快
- 增加网络的非线性
- 使网络具有稀疏性, 减少过拟合

计算图

- 单层神经网络输出后接Relu激活函数

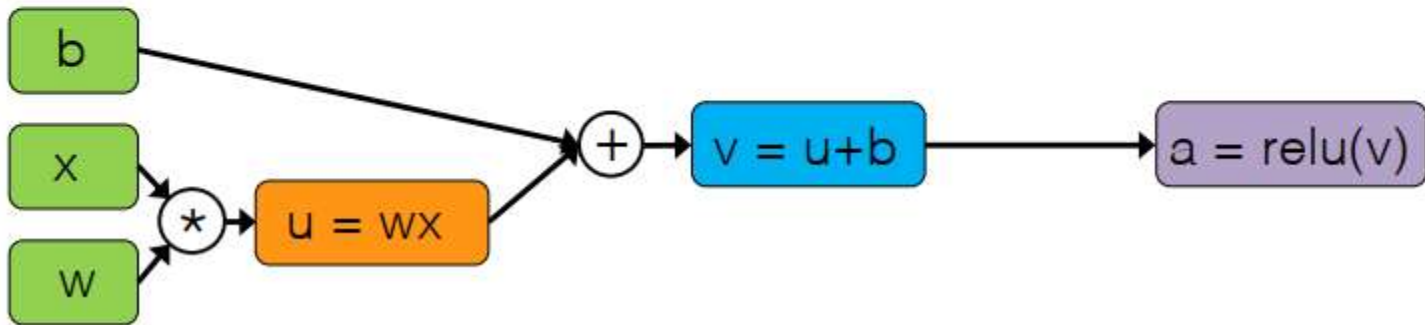


计算图

- 定义了如下计算图
- 前向计算: x 数据; w 、 b 参数

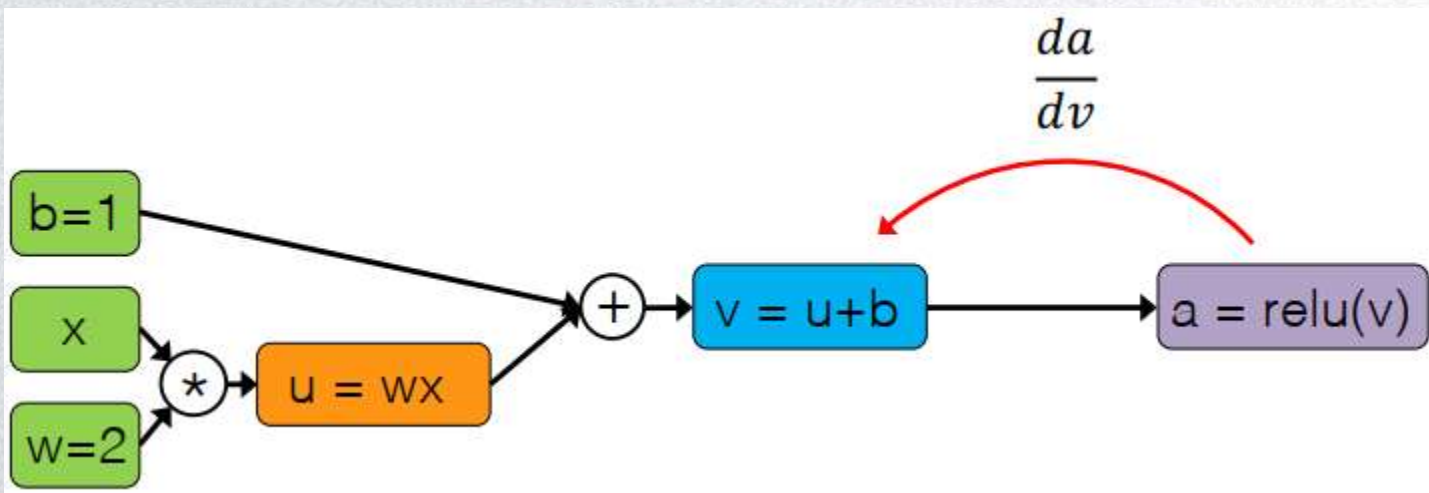
$$a(x, w, b) = \text{relu}(w \cdot x + b)$$

$$\underbrace{\underbrace{w \cdot x}_u + b}_v$$



计算图

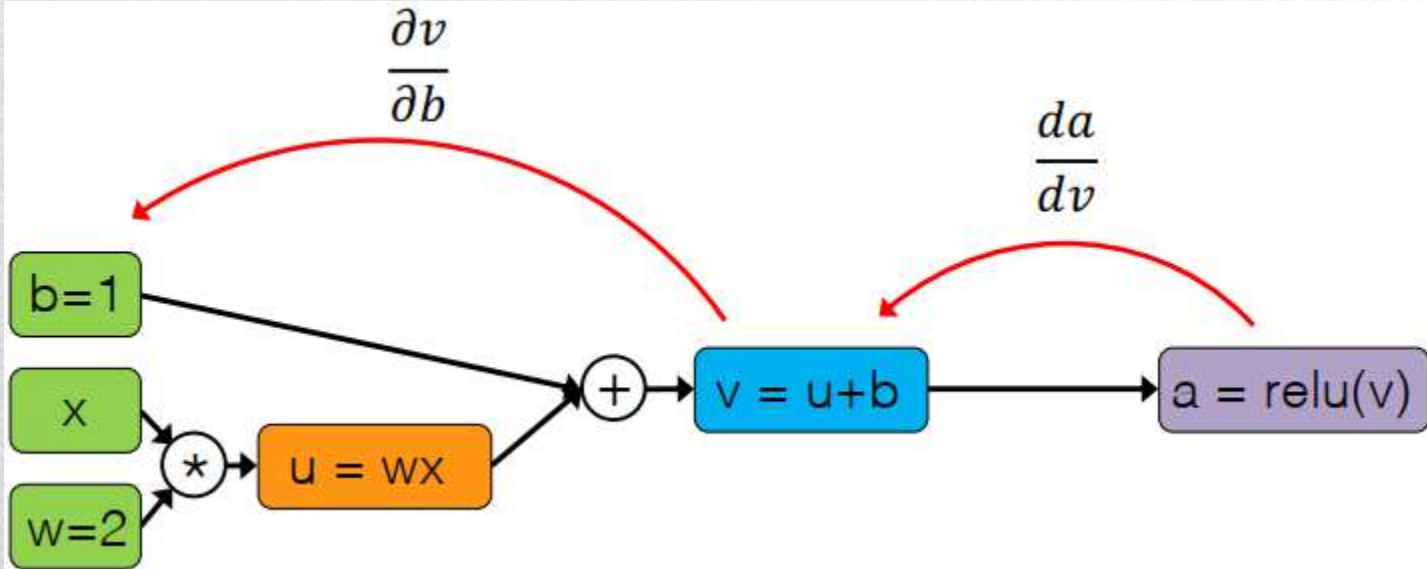
- 深度学习中，我们关心损失函数对参数的导数
- 用链式法则计算激活值对参数的损失
- 反向传播



计算图

- 反向传播

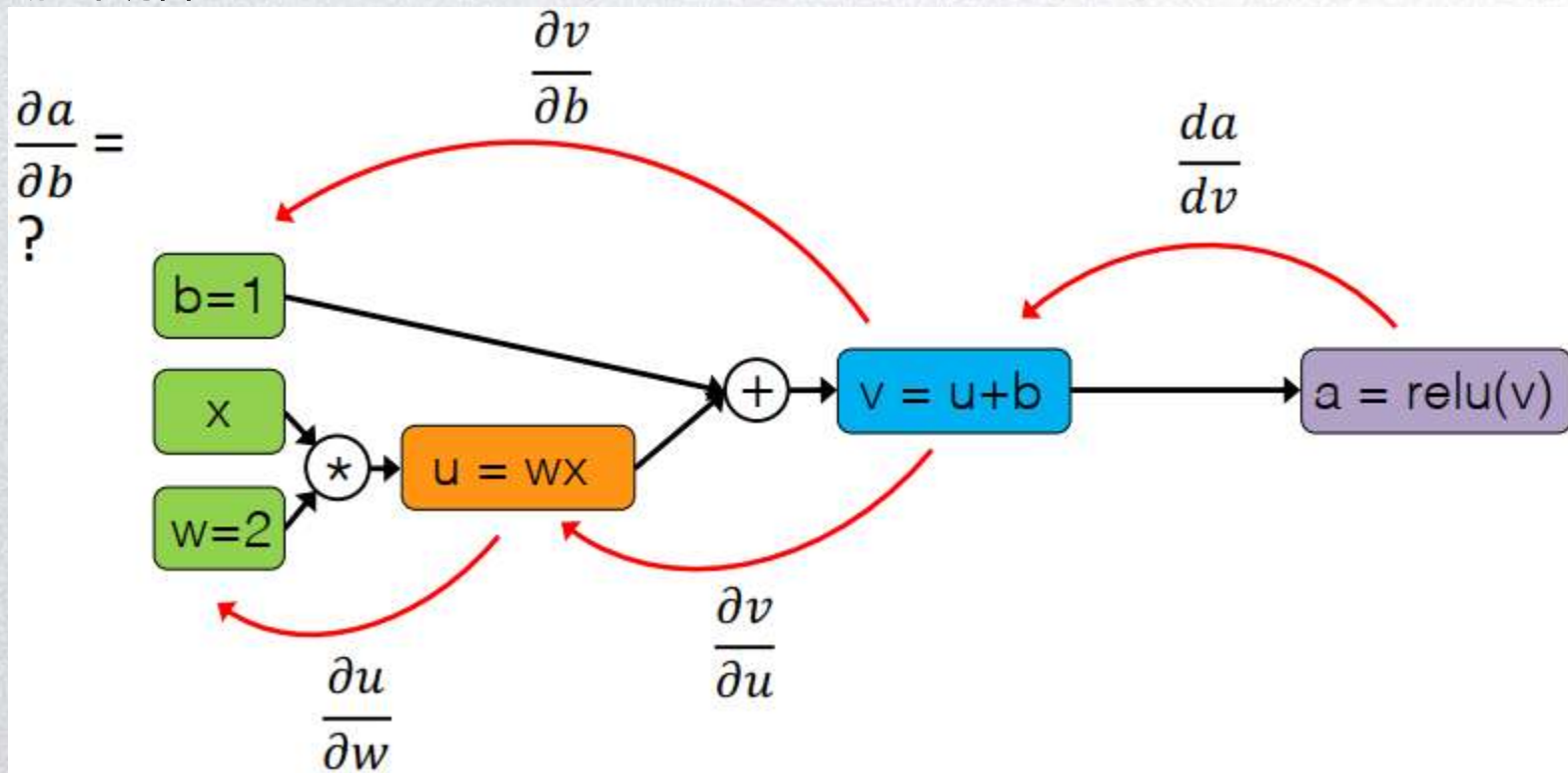
- 一般将用户自己创建的变量是叶子节点，而由叶子节点计算得到的变量是非叶子节点。调用非叶子节点的backward方法，就会沿着非叶子节点一直回溯到叶子节点结束。



计算图



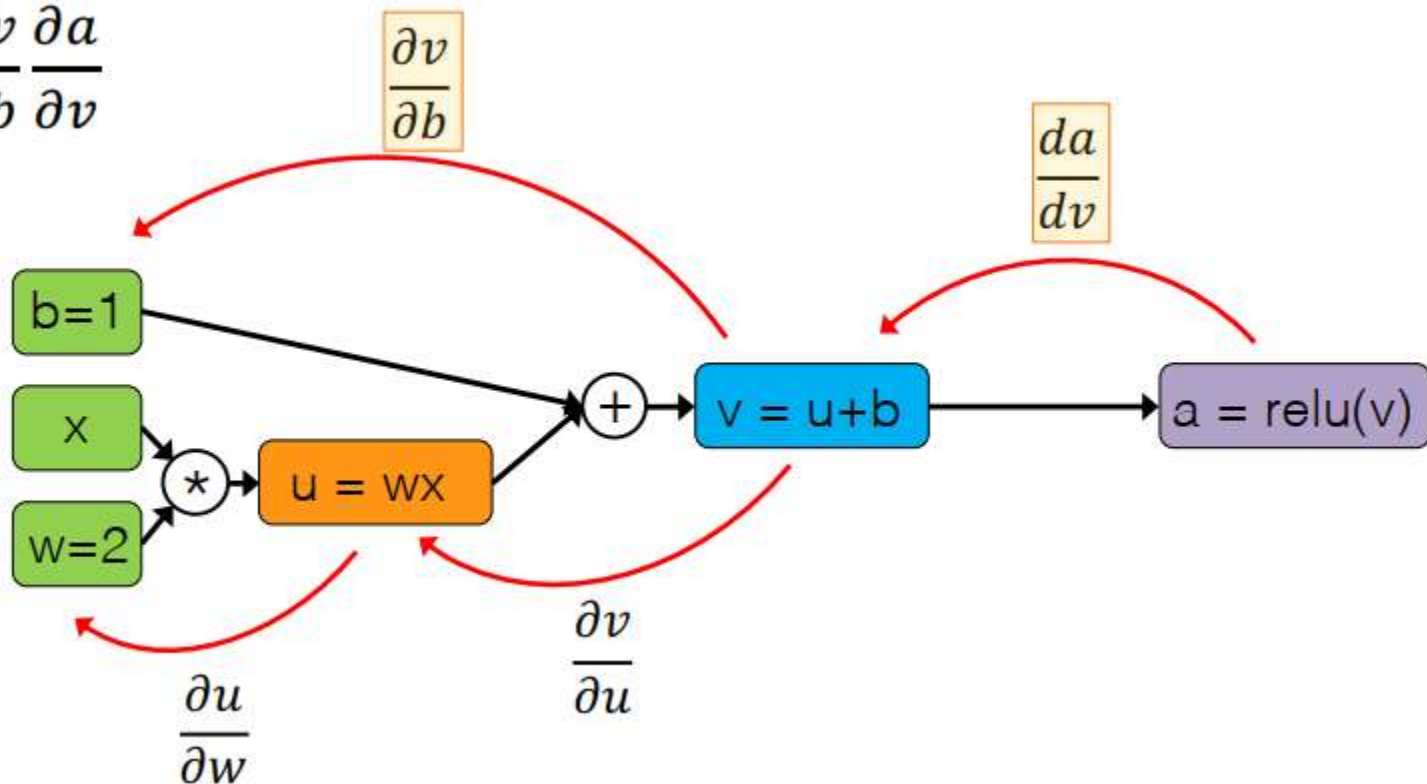
- 反向传播



计算图

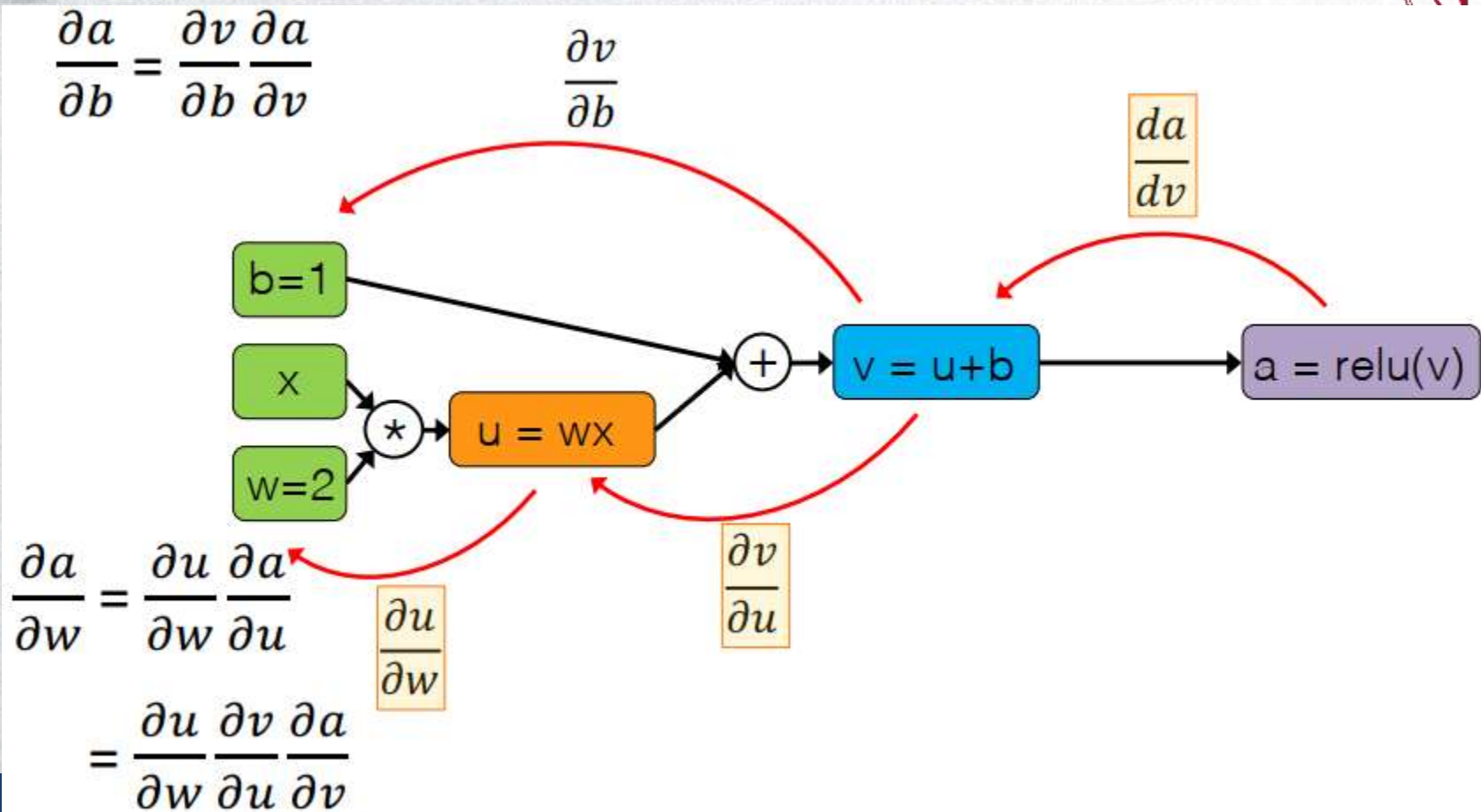


$$\frac{\partial a}{\partial b} = \frac{\partial v}{\partial b} \frac{\partial a}{\partial v}$$



计算图

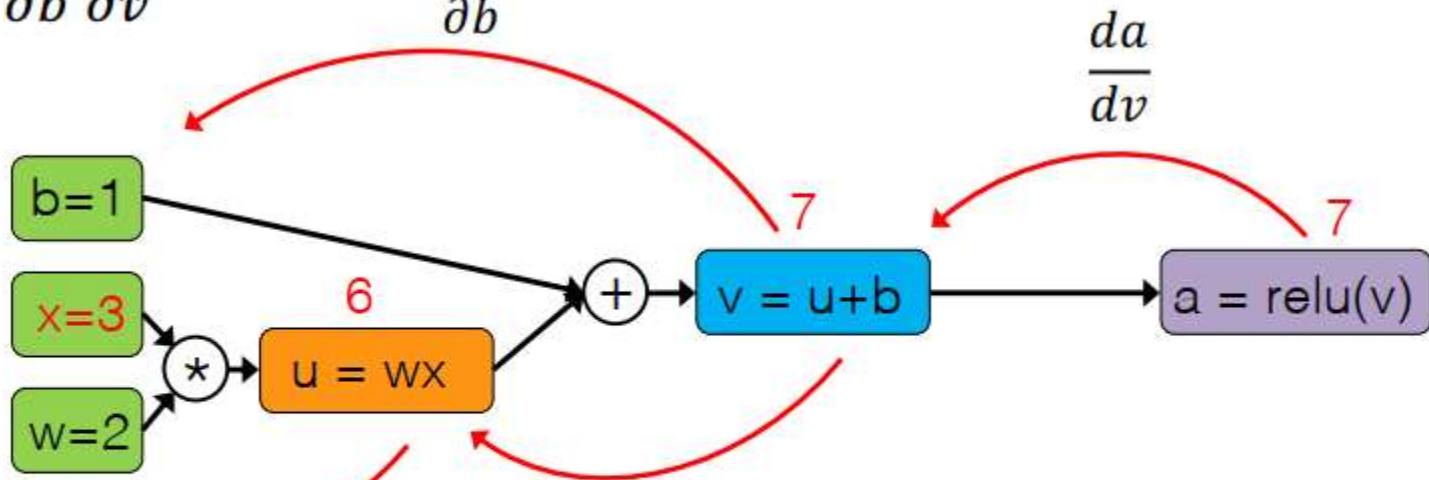
$$\frac{\partial a}{\partial b} = \frac{\partial v}{\partial b} \frac{\partial a}{\partial v}$$



计算图



$$\frac{\partial a}{\partial b} = \frac{\partial v}{\partial b} \frac{\partial a}{\partial v}$$



$$\frac{\partial a}{\partial w} = \frac{\partial u}{\partial w} \frac{\partial a}{\partial u}$$

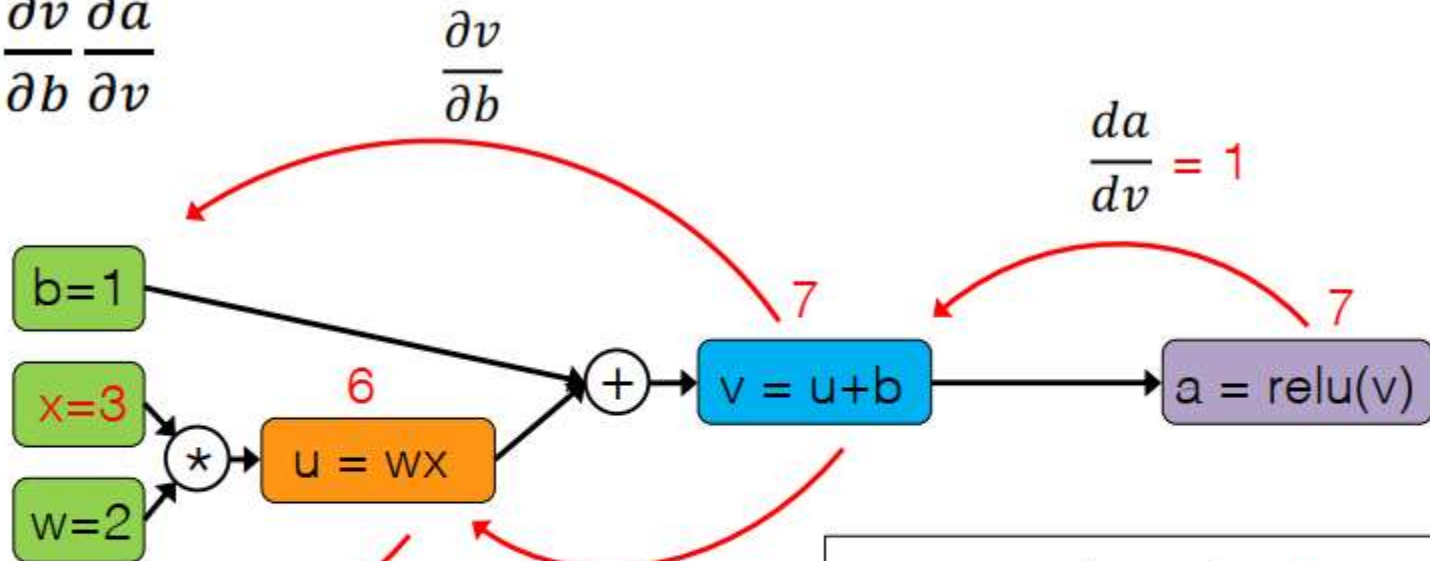
$$\frac{\partial u}{\partial w}$$

$$\frac{\partial v}{\partial u}$$

$$= \frac{\partial u}{\partial w} \frac{\partial v}{\partial u} \frac{\partial a}{\partial v}$$

计算图

$$\frac{\partial a}{\partial b} = \frac{\partial v}{\partial b} \frac{\partial a}{\partial v}$$



$$\frac{\partial a}{\partial w} = \frac{\partial u}{\partial w} \frac{\partial a}{\partial u}$$

$$\frac{\partial u}{\partial w}$$

$$\frac{\partial v}{\partial u}$$

$$= \frac{\partial u}{\partial w} \frac{\partial v}{\partial u} \frac{\partial a}{\partial v}$$

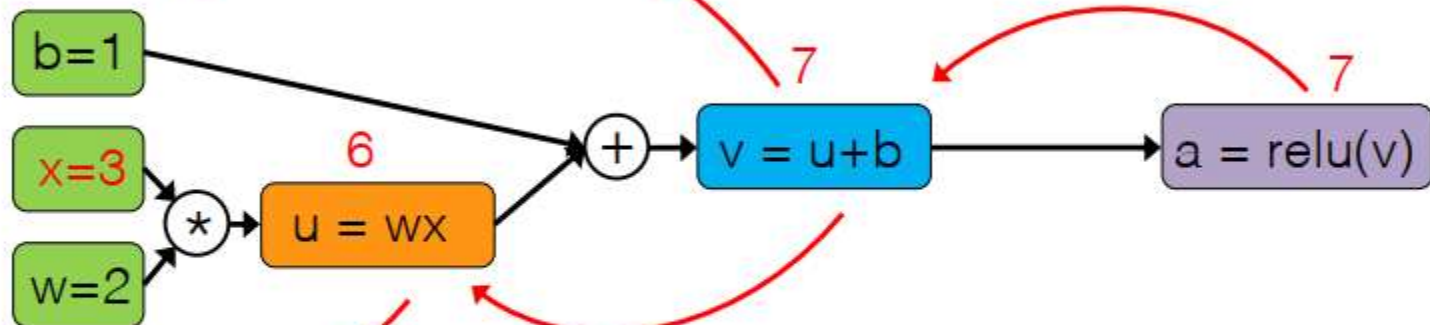
$$\text{relu}(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

计算图

$$\frac{\partial a}{\partial b} = \frac{\partial v}{\partial b} \frac{\partial a}{\partial v}$$

$$\frac{\partial v}{\partial b} = ?$$

$$\frac{da}{dv} = 1$$



$$\frac{\partial a}{\partial w} = \frac{\partial u}{\partial w} \frac{\partial a}{\partial u}$$

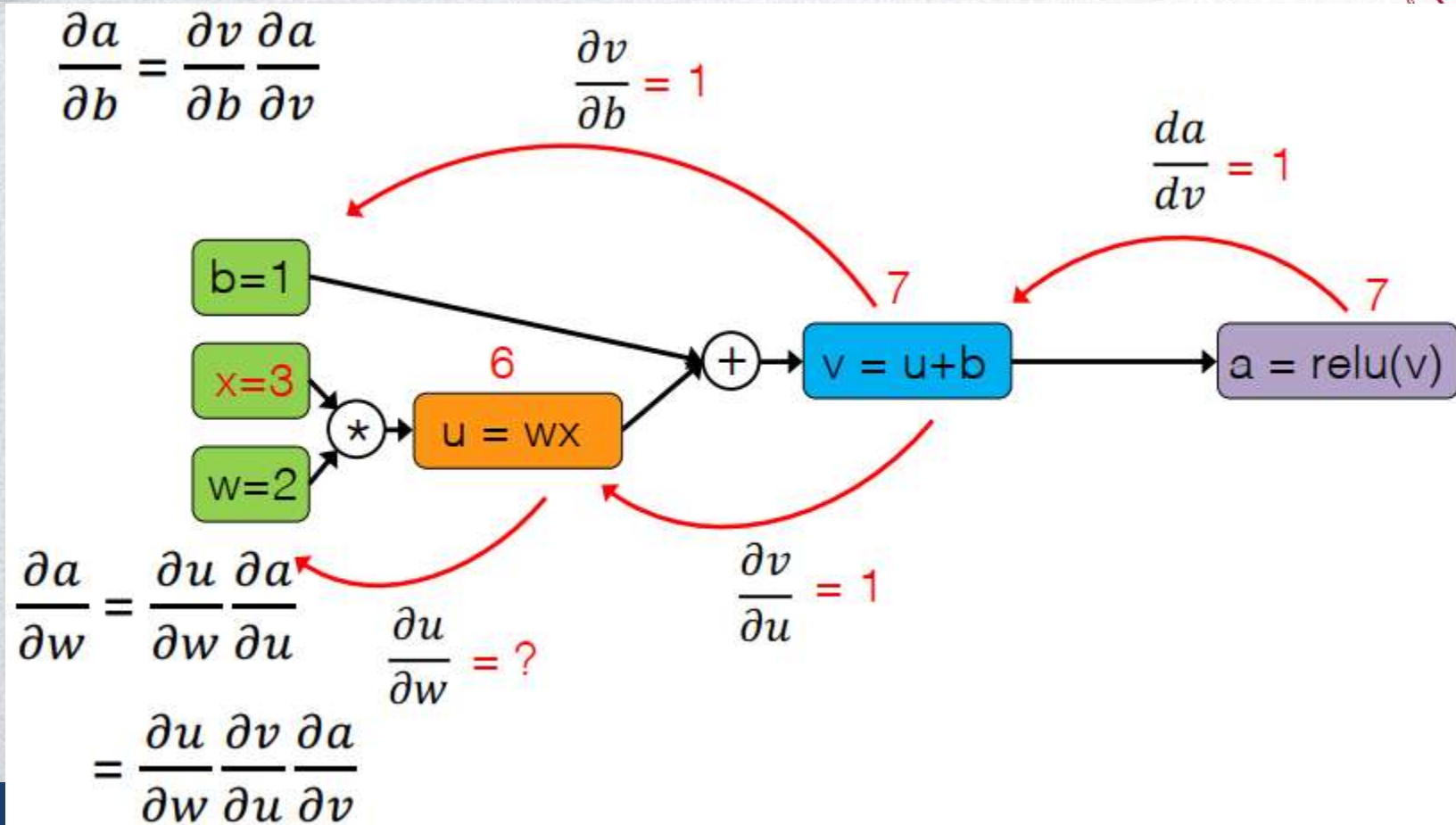
$$\frac{\partial u}{\partial w}$$

$$\frac{\partial v}{\partial u} = ?$$

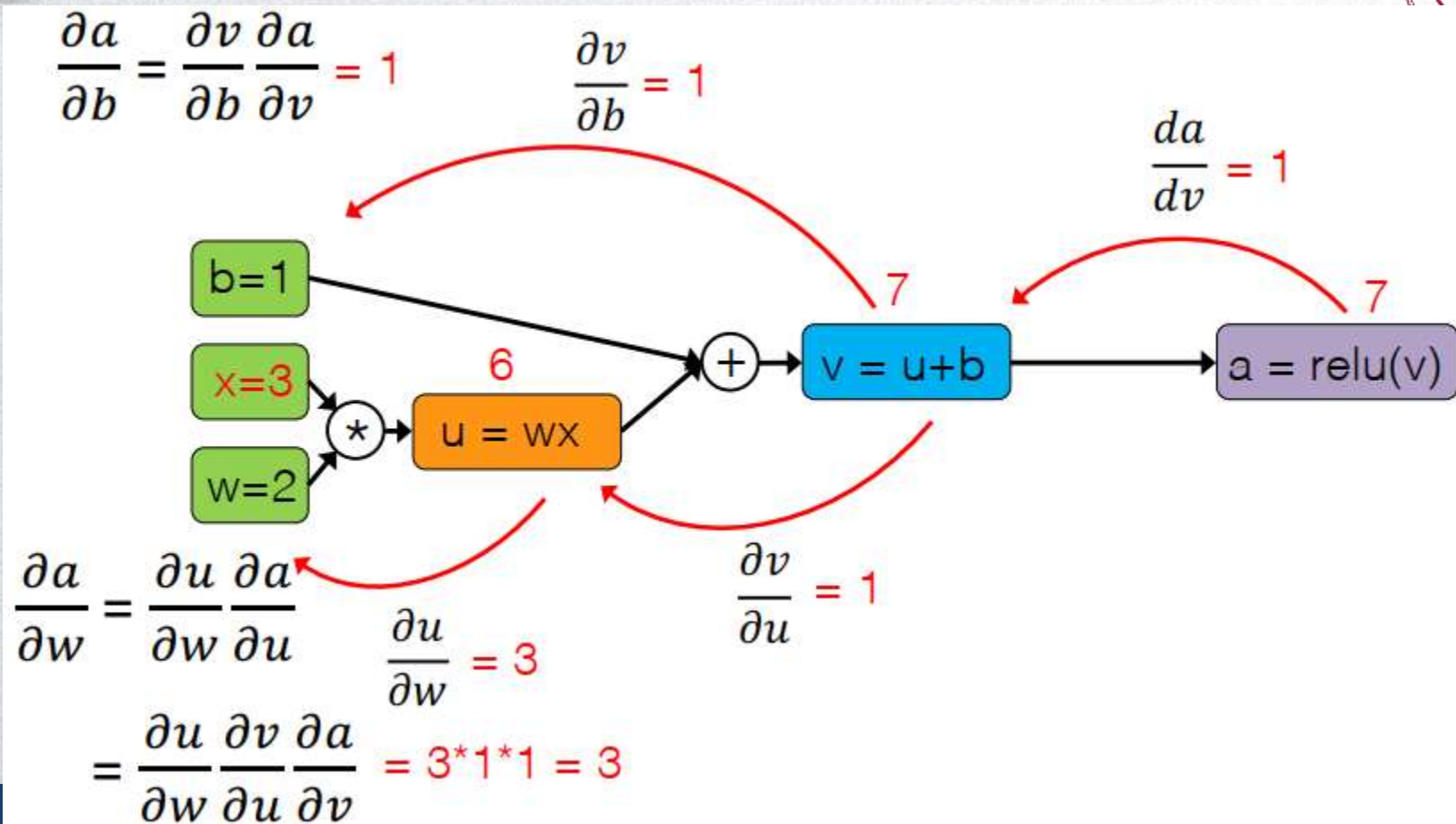
$$= \frac{\partial u}{\partial w} \frac{\partial v}{\partial u} \frac{\partial a}{\partial v}$$

Function	Derivative
$f(x) + g(x)$	$f'(x) + g'(x)$

计算图



计算图





计算图

- y 对 w 求导就是在计算图中找到所有 y 到 w 的路径，把路径上的导数进行求和
- 采用计算图来描述运算的好处不仅仅是让运算更加简洁，更重要的作用是使梯度求导更加方便
 - 叶子节点是整个计算图的根基，例如前面求导的计算图，在前向计算中的 a 、 o 和 l 都要依据创建的叶子节点 x 和 w 进行计算的。同样，在反向传播过程中，所有梯度的计算都要依赖叶子节点。
 - 设置叶子节点主要是为了节省内存，在梯度反向传播结束之后，非叶子节点的梯度都会被释放掉。可以根据代码分析一下非叶子节点 a 、 b 和 y 的梯度情况。
 - 如果想使用非叶子节点梯度，可以使用pytorch中的`retain_grad()`。

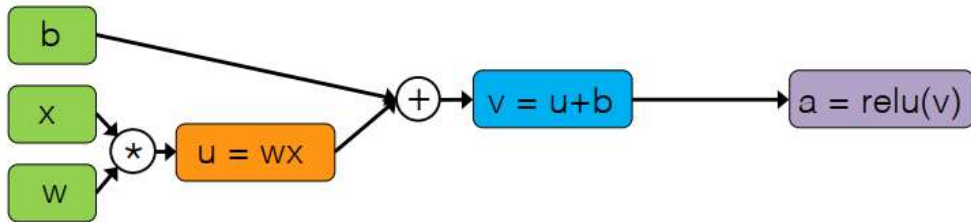


计算图

- Tensor中属性grad_fn的作用是记录创建该张量时所用的方法（函数）
- `u.grad_fn = MulBackward0 object`, `u`在反向传播的时候会记录`u`是用乘法得到的, 所用在求解`w`和`x`的梯度的时候就会用到乘法的求导法则去求解`w`和`x`的梯度。
- 对于`v`有`v.grad_fn= AddBackward0 object`, `v`是通过加法得到的
- 可以通过代码查看tensor的属性
- `v.grad_fn`

$$a(x, w, b) = \text{relu}(\underbrace{w \cdot x}_u + b)$$

v



计算图

```
1 import torch
2 import torch.nn as nn
3
4 x = torch.tensor(3.0, requires_grad=False)
5 w = torch.tensor(2.0, requires_grad=True)
6 b = torch.tensor(1.0, requires_grad=True)
7
8 u = x*w
9 v = u + b
10 F = nn.ReLU(True)
11 a = F(v)
12 a.backward()
13
14 print(w.grad)
15 print(b.grad)
```

```
tensor(3.)
tensor(1.)
```

$$a(x, w, b) = \text{relu}(\underbrace{w \cdot x}_u + b)$$

v

